

Malha de Serviço ({Service Mesh

O que você aprenderá nesta página

Configurar uma malha de serviço (*service mesh*) e usá-la para monitorar o tráfego de rede

Você ouvirá com bastante frequência sobre um conceito chamado *Service Mesh* (Malha de Serviço). As malhas de serviço são animais bastante complexos que fornecem um amplo conjunto de recursos para as aplicações. Durante os capítulos anteriores, implementamos algumas funcionalidades que as malhas de serviço ofereceriam prontamente (*out of the box*).

No contexto da arquitetura de microsserviços, uma malha de serviço é de fato uma ferramenta poderosa que pode agilizar e aprimorar a comunicação entre os serviços. Ela fornece capacidades robustas, como:

proteger a comunicação gerenciar o tráfego, e monitorar o tráfego enviando logs e métricas para ferramentas de observabilidade, como o Prometheus.

Ao usar uma malha de serviço como o Istio, muitas das funcionalidades que foram implementadas manualmente em capítulos anteriores podem ser automatizadas e otimizadas, reduzindo potencialmente a complexidade e a sobrecarga.

O **Istio** é uma popular malha de serviço de código aberto que oferece recursos avançados para gerenciar as interações de serviço a serviço. Ele fornece segurança, controle e observabilidade em todos os microsserviços sem exigir alterações no código da aplicação.

Recentemente, o Istio introduziu um novo recurso chamado *Ambient Mode*, que visa simplificar a implantação e o gerenciamento da malha de serviço. O Ambient Mode reduz a complexidade e a sobrecarga associadas aos tradicionais proxies sidecar adotando uma arquitetura sem sidecars (*sidecar-less*). Este modo permite uma integração mais fácil e reduz o consumo de recursos, tornando-o mais eficiente e escalável para ambientes nativos da nuvem modernos.

Nos próximos exercícios, você se familiarizará com o Istio e seu *ambient mode*.

Leia agora a documentação oficial sobre Istio, *dataplane-modes* e *ambient overview*. Instale agora o Istio CLI através da documentação e configure o Istio no seu cluster k3d seguindo os passos de pré-requisitos. Faça o *deploy* da aplicação de exemplo (Sample app) e siga os passos até o "Clean up" (limpeza).

Nota: é necessário ter o Prometheus instalado no seu cluster. Muito provavelmente você precisará configurar a URL do Prometheus no arquivo *kiali.yaml* da seguinte forma:

```
prometheus:
  enabled: true
  url: http://prom-prometheus-server.monitoring:80
```

Explorar a aplicação de exemplo do Istio mostra bem alguns dos recursos disponíveis, mas não esclarece exatamente como configurar o Istio Ambient para a sua própria aplicação. Portanto, vamos analisar um exemplo muito simples.

Vamos agora implantar uma aplicação *hello world* no *namespace* padrão onde o Istio já está habilitado. O Deployment e o Service são definidos da seguinte forma:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-dep
```

```

spec:
  replicas: 1
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello
    spec:
      containers:
        - name: seedimage
          image: mluukkai/hello:1

---

apiVersion: v1
kind: Service
metadata:
  name: hello-svc
spec:
  type: ClusterIP
  selector:
    app: hello
  ports:
    - port: 80
      protocol: TCP
      targetPort: 8080

```

A definição do Gateway e do HTTPRoute é direta:

```

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: hello-gateway
spec:
  gatewayClassName: istio
  listeners:
    - name: http
      port: 80
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: Same

---

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: hello
spec:
  parentRefs:
    - name: hello-gateway
  rules:
    - matches:
        - path:
            type: Exact
            value: /
      backendRefs:
        - name: hello-svc
          port: 80

```

Após gerarmos algum tráfego para a aplicação, já podemos visualizá-lo com o Kiali.

Vamos agora alterar nossa configuração para uma implantação canário (*canary deployment*). Definimos o *deployment* e o *service* da versão antiga e da nova versão (veja os arquivos de exemplo para *hello-dep-v1*, *hello-svc-v1*, *hello-dep-v2*, *hello-svc-v2*).

Observe como o rótulo (*label*) `version` é usado para distinguir entre as versões! Agora podemos simplesmente alterar o recurso `HTTPRoute` para dividir o tráfego entre os serviços:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: hello
spec:
  parentRefs:
  - name: hello-gateway
  rules:
  - matches:
    - path:
        type: Exact
        value: /
    backendRefs:
    - name: hello-svc-v1
      port: 80
      weight: 75
    - name: hello-svc-v2
      port: 80
      weight: 25
```

O Kiali visualiza isso muito bem e, após gerar algum tráfego, podemos verificar a proporção que cada *pod* recebe.

Existem algumas operações interessantes de gerenciamento de tráfego que ainda não estão disponíveis com as novas ferramentas e exigem o uso do antigo recurso `VirtualService`. Felizmente, podemos usar `VirtualServices` se mudarmos do Gateway API para o recurso **Istio Ingress Gateway**.

Primeiro, devemos instalar o Ingress Gateway seguindo a documentação. Em seguida, podemos substituir o Kubernetes Gateway e `HTTPRoute` pelos recursos Istio Gateway e `VirtualService`:

```
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: hello-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"
---
```

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: helloworld
spec:
  hosts:
  - "*"
  gateways:
  - hello-gateway
  http:
  - match:
    - uri:
        exact: /
    route:
    - destination:
        host: hello-svc-v1
        port:
          number: 80
        weight: 50
```

```
- destination:
  host: hello-svc-v2
  port:
    number: 80
  weight: 50
```

Observe que o Gateway agora tem a versão de API `networking.istio.io/v1`, enquanto o Gateway Kubernetes anterior tinha a versão de API `gateway.k8s.io/v1`. Portanto, apesar de ambos terem o tipo Gateway, esses são recursos completamente diferentes e de forma alguma compatíveis entre si.

Podemos acessar nosso aplicativo fazendo um *port-forward* através do *ingress-gateway* do Istio:

```
kubectrl port-forward svc/istio-ingressgateway 8080:80 -n istio-ingress
```

A definição do VirtualService é bastante semelhante à do HTTPRoute, apenas a sintaxe é um pouco diferente. Um recurso interessante disponível ao usar o VirtualService é a injeção de falhas (*fault injection*), que podemos usar para simular vários tipos de condições de erro. A configuração a seguir define que há 50% de chance de a resposta ser atrasada em 3 segundos e 25% de chance de a requisição ser abortada com um erro HTTP 500:

```
fault:
  delay:
    percentage:
      value: 50
    fixedDelay: 3s
  abort:
    httpStatus: 500
    percentage:
      value: 25
```

Implemente a aplicação Log na malha de serviço (*service mesh*). Estenda a aplicação com um novo serviço *greeter* que responda a uma requisição HTTP GET com uma saudação (como *hello*). A aplicação *log output* deve usar o serviço *greeter* e exibir a saudação juntamente com o restante da saída.

Assim que a configuração básica estiver funcionando, faça o *deploy* de duas versões do *greeter* e divida o tráfego de modo que uma versão receba 75% e a outra o restante.

Dicas:

Nossa configuração é muito semelhante à usada na aplicação de exemplo do Istio. Como você verá, a divisão de tráfego é feita com um objeto HTTPRoute, onde você precisa definir um serviço (o *greeter-svc*) como um `parentRefs`. Você precisará de três serviços separados para o *greeter*: o *greeter-svc* ao qual você anexa o HTTPRoute, que apontará para o *greeter - svc - 1* e *greeter - svc - 2*.

Use o Kiali para garantir que o tráfego seja dividido corretamente entre os *greeters*.

Mais uma coisa... init containers e sidecars

Por debaixo dos panos, muitas implementações de malha de serviço dependem fortemente de contêineres de inicialização (*init containers*) e contêineres secundários (*sidecar containers*) para realizar a sua magia. Para o Istio, a arquitetura baseada em *sidecars* costumava ser a única opção antes da introdução do *ambient mode*, que utilizamos em exercícios anteriores. Embora os *sidecars* não sejam mais a única opção para implementar uma malha de serviço, eles permanecem um conceito útil com diversas aplicações. Vamos dar uma olhada mais de perto nesses conceitos muito importantes.

Um *pod* pode ter qualquer número de contêineres de inicialização (*init containers*), que são contêineres que rodam antes de os contêineres principais do *pod* iniciarem. Existem muitos usos para eles, como por exemplo:

gerar ou modificar arquivos de configuração, buscar dados ou configurações de fontes remotas ou realizar qualquer pré-processamento necessário *antes* do início da aplicação principal aguardar que outros serviços, bancos de dados ou componentes de infraestrutura estejam operacionais *antes* do início da aplicação principal. Isso garante que os contêineres principais iniciem apenas quando todas as dependências estiverem prontas instalar ou configurar utilitários, cadeias de ferramentas ou software exigidos pela aplicação principal durante a execução, mas não incluídos na imagem principal, mantendo a imagem principal leve e otimizada

Os *sidecars* são contêineres secundários que rodam junto ao contêiner da aplicação principal dentro do mesmo *Pod*. Esses contêineres são usados para aprimorar ou estender a funcionalidade do contêiner da aplicação principal, fornecendo serviços ou funcionalidades adicionais como registro (*logging*), monitoramento, segurança ou sincronização de dados, sem alterar diretamente o código da aplicação principal.

Escreva um aplicativo que sirva páginas da Wikipedia. O aplicativo deve conter:

O contêiner principal baseado na imagem *nginx*, que simplesmente sirva qualquer conteúdo que esteja no diretório público *www*. Um *init container* que execute um `curl` na página `{https://en.wikipedia.org/wiki/Kubernetes}` e salve o conteúdo no diretório público *www* para o contêiner principal. Um *sidecar container* que aguarde um tempo aleatório entre 5 e 15 minutos, execute um `curl` em uma página aleatória da Wikipedia usando a URL `{https://en.wikipedia.org/wiki/Special:Random}` e salve o conteúdo no diretório público *www* para o contêiner principal.